Seven Principles of Good Relational Database Design

1. Normalize to Eliminate Redundancy

Normalization is the process of organizing columns and tables so that each fact is stored in exactly one place. When the same piece of data is duplicated across rows, you risk **update anomalies** — situations where updating one copy but not another leaves your database internally contradictory.

Consider this single, "flat" table:

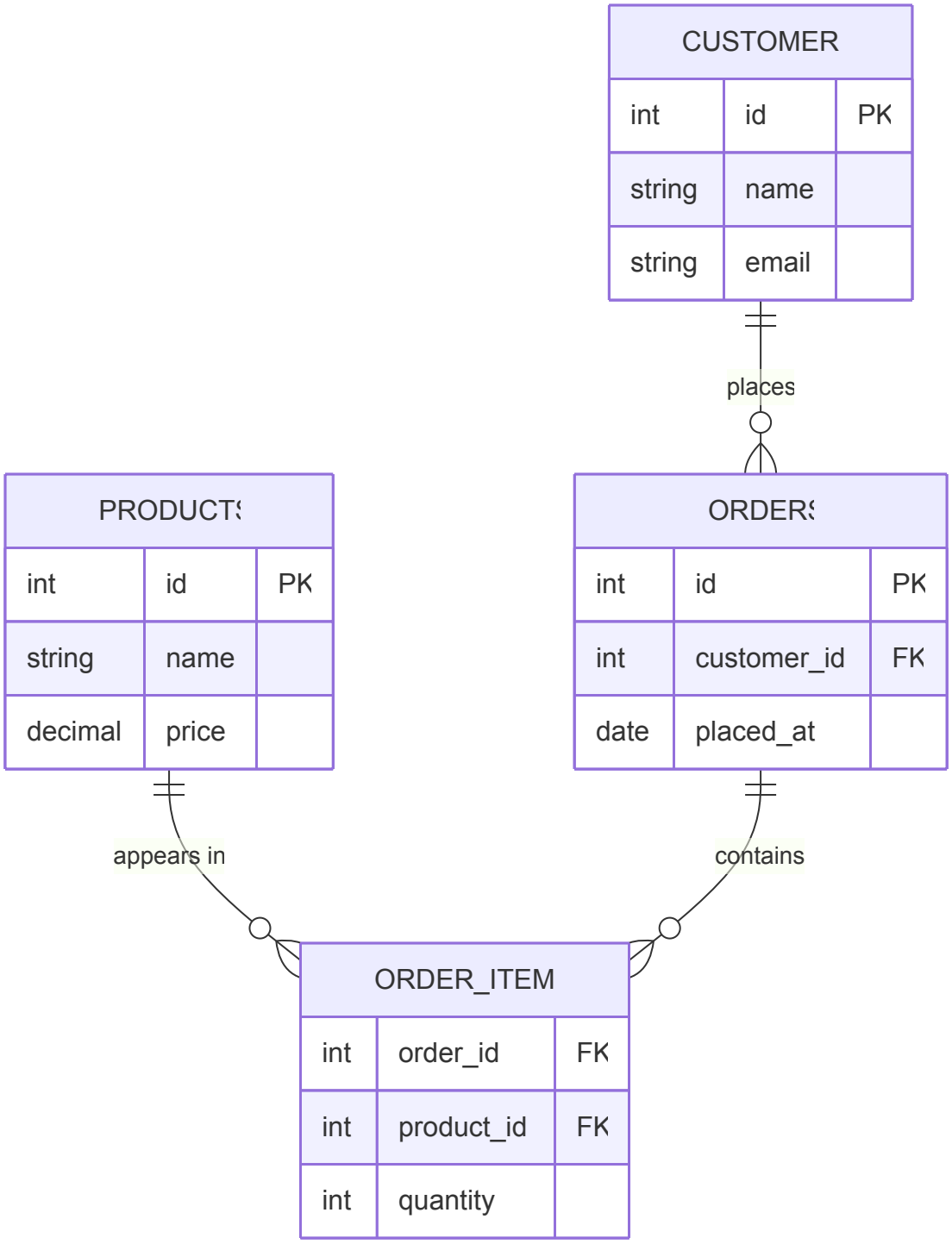
order_id	customer_name	customer_email	product_name	product_price
1	Ana Ruiz	ana@mail.com	Widget	9.99
2	Ana Ruiz	ana@mail.com	Gadget	14.99
3	Jon Park	jon@mail.com	Widget	9.99

Ana's email is stored twice. If she changes it, you must remember to update both rows — miss one, and you now have two "true" emails for the same person. This violates **Third Normal Form (3NF)**, the practical target for most application databases.

The normalization ladder, briefly:

- **1NF (First Normal Form):** each column holds a single, atomic value (no comma-separated lists crammed into one cell).
- **2NF (Second Normal Form):** every non-key column depends on the *whole* primary key, not just part of it (matters for tables with composite keys).
- **3NF (Third Normal Form):** every non-key column depends on the key **and nothing but the key** — no column depends on another non-key column.

Fixing the table above means splitting it into three:



Now Ana's email lives in exactly one row. Update it once, and every order automatically reflects the change through the foreign key relationship.

> **Rule of thumb:** if you ever catch yourself writing an `UPDATE` statement that touches many rows to fix "the same" piece of information, that's a sign the data isn't normalized.

---

2. Choose Your Keys Deliberately

Not all primary keys are created equal, and the choice affects performance, referential stability, and how easy your schema is to reason about.

Surrogate key: an artificial identifier with no business meaning — typically an auto-incrementing integer or a UUID — created solely to identify a row.

Natural key: a column (or set of columns) that already exists in the real world and uniquely identifies the entity, such as an email address, a national ID number, or an ISBN.

	Surrogate key (e.g. `id INTEGER PRIMARY KEY`)	**Natural key (e.g. `email TEXT PRIMARY KEY`)**
Stability	Never changes	Can change (people update emails)
Join performance	Small, fast to index	Often larger/slower to index (text vs int)
Meaning	Meaningless outside the DB	Self-documenting
Risk	None inherent	Business rules change — "unique" things turn out not to be

General guidance: default to a surrogate integer primary key for most tables. Reserve natural keys for genuinely immutable, verified identifiers (e.g., an ISO country code), and even then, many teams still add a surrogate key and enforce the natural key as a `UNIQUE` constraint instead — you get stability *and* a data-integrity guarantee.

A **composite key** (a primary key made of two or more columns together) is appropriate when uniqueness only makes sense in combination — like the `order_items` table above, where `(order_id, product_id)` together identify a line item, but neither column alone does.

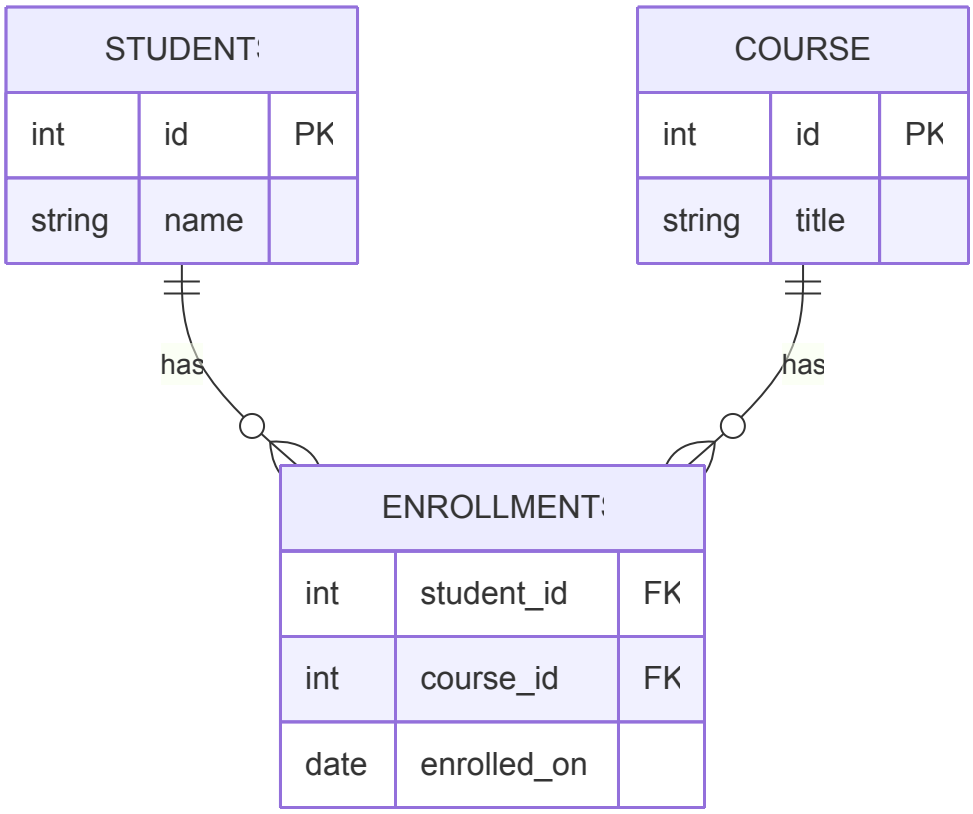
---

3. Model Relationships With the Right Cardinality

Cardinality describes how many rows in one table can relate to how many rows in another: one-to-one, one-to-many, or many-to-many. Getting this wrong is one of the most common design mistakes, because it's invisible until the data grows.

- **One-to-many** (most common): one customer, many orders. Modeled with a foreign key on the "many" side, pointing back to the "one" side.
- **One-to-one**: rare — usually means you're splitting a table for organizational or security reasons (e.g., a `users` table and a separate `user_profiles` table with sensitive extra fields).

- **Many-to-many**: many students can enroll in many courses. This *cannot* be modeled with a single foreign key column — it requires a **junction table** (also called an associative or join table): a table whose whole job is to record valid pairings.



A frequent beginner mistake is trying to store multiple course IDs in a single comma-separated column on the `students` table. That breaks 1NF (see Section 1), makes filtering and joining nearly impossible with plain SQL, and prevents the database from enforcing referential integrity at all.

---

4. Enforce Data Integrity With Constraints

A **constraint** is a rule the database itself enforces, so invalid data can never be saved — regardless of which application, script, or careless developer is writing to it. Relying on your application code alone to "always remember" to validate data is fragile; constraints make invalid states physically impossible to insert.

Key constraint types beyond primary/foreign keys:

- `NOT NULL` — a column must always have a value (e.g., an order should never lack a `customer_id`).
- `UNIQUE` — no two rows may share the same value in this column (e.g., usernames or emails).

- **`CHECK`** — a custom condition a row must satisfy, e.g. `CHECK (price >= 0)` or `CHECK (status IN ('pending','shipped','cancelled'))`.
- **`DEFAULT`** — a fallback value applied automatically when none is given, e.g. `DEFAULT CURRENT_TIMESTAMP` for a `created_at` column.
- **`ON DELETE` / `ON UPDATE` rules on foreign keys** — what should happen to child rows when a parent row is deleted. Common options:
  - `CASCADE` — delete/update child rows automatically
  - `RESTRICT` / `NO ACTION` — block the delete if children exist
  - `SET NULL` — clear the foreign key on child rows

```
CREATE TABLE orders (
    id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL REFERENCES customers(id) ON DELETE
RESTRICT,
    status TEXT NOT NULL DEFAULT 'pending' CHECK (status IN
('pending','shipped','cancelled')),
    placed_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Think of constraints as documentation that enforces itself: anyone reading this `CREATE TABLE` statement instantly knows the business rules, and no code path can silently violate them.

---

5. Pick the Right Data Type for Each Column

SQLite is famously permissive about types, which makes it easy to develop bad habits that hurt you in stricter databases (Postgres, MySQL) or cause subtle bugs even in SQLite itself.

Common pitfalls and better choices:

Instead of...	Use...	Why
Storing dates as free-text strings in mixed formats	A consistent format, e.g. ISO 8601 (`'2026-09-14'`)	Sorts and compares correctly as text; avoids ambiguity like `03/04/26`
Storing money as `FLOAT` / `REAL`	An integer of the smallest unit (cents) or a fixed-point `DECIMAL` type	Floating-point math introduces rounding errors — bad news for money
A `status` column as free text with no constraint	Text restricted with a `CHECK` constraint, or a	Prevents typos like `'shiped'` from silently creating a new,

Instead of...	Use...	Why
	lookup table (see below)	invalid status
Cramming multiple values into one column (`"red,blue,green"`)	A separate related table (one row per value)	Keeps data queryable and atomic (1NF again)
Using `NULL` and `''` (empty string) interchangeably to mean "no value"	Pick one convention — usually `NULL` — and enforce it	Two different empty-value formats will produce baffling bugs (e.g. `''` vs `NULL` do not match in `WHERE` filters)

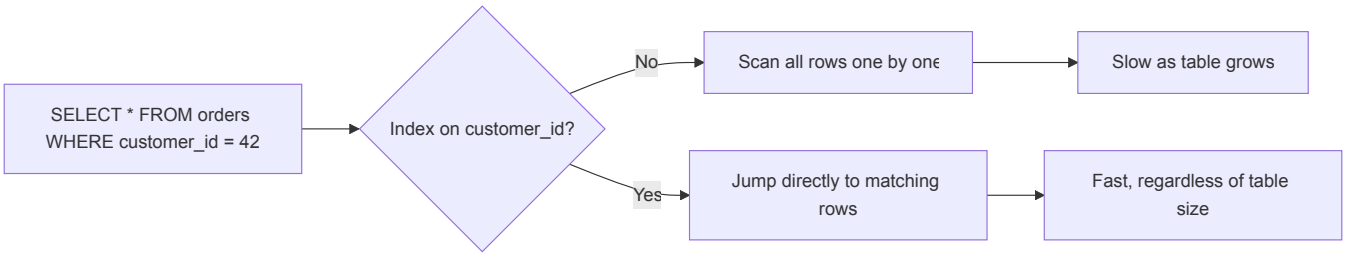
A **lookup table** (sometimes called a reference or enum table) is a small table that holds the fixed set of valid values for a column, referenced by foreign key — e.g. an `order_statuses` table with rows for `pending`, `shipped`, `cancelled`. This is often preferable to a `CHECK` constraint when the list of valid values might grow, or when each value needs extra metadata (like a display label or sort order).

---

6. Index for the Queries You Actually Run

An **index** is a separate, sorted data structure the database maintains alongside a table, so it can find matching rows quickly without scanning every row — much like the index at the back of a book. Primary keys are indexed automatically; most other columns are not, by default.

Without an index, filtering on a column forces a **full table scan**: the database checks every single row. On a table with a hundred rows, you'll never notice. On a table with ten million, that query goes from milliseconds to seconds.



Practical guidance:

- **Index foreign key columns.** SQLite does not do this automatically, and joins/filters on foreign keys are extremely common.
- **Index columns you frequently filter or sort by** (`WHERE`, `ORDER BY`), not every column — each index speeds up reads but slows down writes (`INSERT` / `UPDATE` / `DELETE`), since the index must be updated too.

- **Composite indexes** (an index across multiple columns) help when you consistently filter by the same combination, e.g. `(customer_id, placed_at)` for "this customer's orders, sorted by date."
- Don't index columns with very low **cardinality** (few distinct values, like a boolean `is_active` flag) in isolation — the index rarely narrows the search enough to be worth the write overhead.

Indexing is the one principle that's genuinely a trade-off rather than a free win — the goal is matching indexes to your actual query patterns, not maximizing the count.

---

7. Know When (and When Not) to Denormalize

Section 1 made the case for normalization. But normalization isn't a moral rule — it's a tool for correctness, and it comes at a cost: retrieving a complete picture of "an order with its customer and products" now requires joining four tables instead of reading one row.

Denormalization is the deliberate, controlled reintroduction of redundancy to improve read performance or simplify queries — usually applied *after* you've confirmed a normalized design is a genuine bottleneck, not as a starting point.

Common, reasonable denormalization patterns:

- **Cached aggregates:** storing an `order_count` column on `customers` instead of running `COUNT(*)` over `orders` every time, updated whenever an order is placed or removed.
- **Snapshotting at time of transaction:** storing `product_name` and `product_price` directly on `order_items` at the moment of purchase, in addition to the foreign key. This isn't sloppy design — it's correct! Historical orders shouldn't retroactively change if you later rename or reprice a product.
- **Read-heavy reporting tables:** a separate, wide "denormalized" table rebuilt periodically for dashboards, kept apart from the normalized tables that handle everyday writes.

The key discipline: denormalize intentionally and document *why*, keep the normalized source of truth intact underneath, and have a clear plan (a trigger, a scheduled job, application logic) for keeping the redundant copy in sync. Denormalization adopted by accident — because joining tables felt inconvenient — is how you end up back at Section 1's problem.

---

8. Ten Red Flags to Watch For

If you spot any of these in a schema — yours or someone else's — treat it as a prompt to stop and reconsider the design:

1. **A column holding comma-separated or JSON-blob values that should be their own rows** (e.g. `tags: "sale,clearance,new"`). Almost always a sign a related table is missing.
2. **No foreign key constraints between clearly related tables** — relationships enforced only "in the application," which means one buggy script can orphan or corrupt data.
3. **A `type` or `category` column with no `CHECK` constraint and no lookup table** — nothing stops `'active'`, `'Active'`, and `'ACTIVE'` from all existing simultaneously.
4. **Multiple columns like `product_id_1`, `product_id_2`, `product_id_3`** on one table — a hard cap on a relationship that should be one-to-many or many-to-many via a junction table.
5. **Duplicated data with no clear source of truth** — the same customer's address stored (and drifting out of sync) in three different tables.
6. **Primary keys that are meaningful business data prone to change** (e.g. using `email` as a primary key that other tables reference by foreign key, then having no plan for what happens when someone changes their email).
7. **Money stored as floating-point** (`FLOAT` / `REAL`) rather than integer cents or fixed-point decimal.
8. **No `created_at` / `updated_at` timestamps** on tables that track evolving state — makes debugging, auditing, and "what changed and when" nearly impossible after the fact.
9. **Every column allows `NULL`**, including ones that logically must always have a value — usually a sign constraints were never thought through, not that the data is genuinely optional everywhere.
10. **Indexes added reflexively to every column** ("just in case") without regard to actual query patterns — bloats storage and silently slows down every write.

None of these are automatically fatal — real-world schemas sometimes have good reasons for an apparent shortcut. But each one should make you ask "why," and be able to answer it.